

Data Science

A Complete Guide

The full data science workflow: data, statistics, machine learning, time series, NLP, deployment, and career — with Python

1. Introduction to Data Science

CHAPTER 1

What is Data Science?

Data science is the discipline of extracting insight and value from data through a combination of statistics, programming, domain knowledge, and communication. It turns raw data into decisions: predictions, recommendations, dashboards, and products.

Data science sits at the intersection of three skills:

- **Mathematics & Statistics** — the theory of uncertainty, distributions, and inference.
- **Programming & Engineering** — collecting, cleaning, and modeling data with code (mostly Python).
- **Domain & Business Knowledge** — knowing which problems matter and what a good answer looks like.

That last piece is why data science is not just "statistics with Python." A model that predicts perfectly but answers the wrong question is worthless. The best data scientists spend as much time framing the problem as building the model.

Data Science vs Related Fields

Field	Focus	Primary Output
Data Analyst	Describing what happened with reports and dashboards	Dashboards, reports, insights
Business Intelligence	Monitoring KPIs against predefined metrics	Dashboards, alerting
Data Scientist	Explaining why it happened and predicting what happens next	Models, experiments, insights, products
Machine Learning Engineer	Industrializing models into reliable services	Deployed models, pipelines, APIs
Data Engineer	Building the pipelines and storage that feed everyone else	Pipelines, warehouses, lakes
Statistician	Rigorous inference under uncertainty	Analyses, experimental designs

Reality check: job titles blur in practice. A "data scientist" at a startup may do all of the above. The skills in this book prepare you for the whole spectrum; where you land depends on your emphasis.

The Data Science Workflow

Almost every project follows the same loop, known by names such as CRISP-DM (Cross Industry Standard Process for Data Mining):

1. **Understand the problem.** What decision are we informing? Define success concretely (e.g., "reduce customer churn by 10%").
2. **Collect data.** Identify sources, gather samples, and document provenance.
3. **Clean & prepare.** Fix missing values, types, duplicates, and errors; engineer features.
4. **Explore & visualize.** Generate hypotheses, find patterns, and surface data problems.
5. **Model.** Choose algorithms, train, validate, and compare performance.
6. **Interpret & communicate.** Translate results into decisions the business can act on.
7. **Deploy & monitor.** Put the model in production and watch it drift.

You will go through these stages many times per project — the loop is the point, not the order.

The Data Science Lifecycle in a Company

Inside an organization, data science depends on a healthy data ecosystem:

- **Data producers** — transaction systems, apps, IoT devices, third-party feeds.
- **Data platform** — warehouses, lakes, and pipelines built by data engineers.
- **Data scientists** — turn that data into models and analysis.
- **Consumers** — product teams, executives, and external users who act on the output.

Good to know: most real-world projects fail not on modeling but on data quality, unclear objectives, or a mismatch between the model's output and the decision it should drive.

Core Tools of the Trade

Layer	Tools (this book)
Language	Python 3
Interactive notebooks	Jupyter, VS Code
Tabular data	pandas, NumPy
Visualization	matplotlib, seaborn, plotly
Statistics	scipy.stats, statsmodels
Machine learning	scikit-learn, XGBoost
SQL databases	PostgreSQL, SQLite
Version control	Git + GitHub
Cloud & scale	Spark, cloud notebooks (Colab, BigQuery)

What You Will Build

By the end of this course you will have completed an end-to-end portfolio: a cleaned and analyzed dataset, a hypothesis test, a machine learning model, a time series forecast, a text analysis, and a deployment-ready pipeline — each documented and shared on GitHub.

Practice Task: Pick a question you care about (sports, finance, health, gaming, your workplace). Write it as a decision: "If I knew X, I would do Y." List what data would answer it, where that data lives, and how you would measure success. Save this as the opening of your project README.

10. Machine Learning Fundamentals

CHAPTER 10

What Machine Learning Is

Machine learning builds models that learn patterns from data rather than from explicit rules. You give the algorithm examples (input → output) or just inputs, and it finds the mapping that generalizes to new data. The key word is **generalize**: a model that memorizes the training data is useless.

Supervised vs Unsupervised

Type	Labeled?	Goal	Examples
Supervised	Yes (features + target)	Predict the target for new rows	Regression, classification
Unsupervised	No (features only)	Find structure in data	Clustering, dimensionality reduction
Reinforcement	Rewards, not labels	Learn actions that maximize reward	Game AI, recommendation policies

The Train/Test Split

Never evaluate a model on the data used to train it — it will look artificially good. Hold out data the model never saw.

```
from sklearn.model_selection import train_test_split

X = df.drop(columns=["spend"])
y = df["spend"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=None
)
```

Data leakage: if any preprocessing uses information from the test set (e.g., fitting a scaler on train+test together), your evaluation is optimistic and your real-world performance will be worse. Fit scalers and encoders on *training data only*, then transform test data with the same fitted object. Chapter 15's pipelines handle this automatically.

Bias-Variance Trade-off

- **Bias** — error from assuming the model shape is too simple (underfitting).
- **Variance** — error from the model being too sensitive to training noise (overfitting).

The sweet spot balances both. Signs of overfitting: great train scores, poor test scores. Signs of underfitting: both scores are poor.

Cross-Validation

k-fold cross-validation splits the data k times, training on k-1 folds and validating on the remaining one, averaging the scores. It uses your data more efficiently than a single split.

```
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(n_estimators=200, random_state=42)
scores = cross_val_score(model, X_train, y_train, cv=5,
                          scoring="neg_mean_absolute_error")
print(scores.mean(), scores.std())
```

Metrics by Task

Regression:

Metric	Meaning
MAE	Mean absolute error (easy to explain, in target units)
RMSE	Root mean squared error (penalizes big errors)
R ²	Fraction of variance explained (not "accuracy")

Classification:

Metric	Meaning
Accuracy	Correct / total (misleading on imbalanced classes)
Precision	Of predicted positives, how many were right
Recall	Of actual positives, how many were caught
F1	Harmonic mean of precision and recall
ROC-AUC	Ranking quality across thresholds
PR-AUC	Better than ROC-AUC on imbalanced data

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred, target_names=["no", "yes"]))
```

Class Imbalance

When one class is rare (fraud: 0.1%, churn: 2%), accuracy is a trap — a model predicting "no fraud" always is 99.9% "accurate."

- Report precision/recall/F1, not accuracy.
- Use `class_weight="balanced"` in many sklearn models.

- Resample: oversample the minority or undersample the majority (with care).
- Prefer probability-based decisions with a tuned threshold (choose the threshold from the business cost of false positives vs false negatives).

Interpretability vs Performance

Simple models (linear regression, decision trees) are easy to explain; complex models (gradient boosting, neural nets) predict better. For regulated or high-stakes decisions, start simple, and use SHAP / permutation importance to explain complex ones. A model you cannot explain is hard to deploy responsibly (Chapter 17).

Practice Task: On your dataset, pick a numeric target (regression) or a categorical target (classification). Build a baseline model (a simple linear model or logistic regression), set up a train/test split with a preprocessing fit on train only, and report the two most relevant metrics per task. State in one sentence what overfitting would look like for your model and how cross-validation protects you.

11. Regression & Classification with scikit-learn

CHAPTER 11

The scikit-learn Workflow

Every supervised model in scikit-learn follows the same pattern:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X = df[["age", "tenure", "support_tickets"]]
y = df["spend"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LinearRegression())
])
pipeline.fit(X_train, y_train)

y_pred = pipeline.predict(X_test)
```

Why pipelines: the pipeline fits the scaler on training data only and reuses it on test data — no leakage (Chapter 10). It also bundles preprocessing and model into one object you can save, cross-validate, and deploy.

Linear Regression

```
from sklearn.metrics import mean_absolute_error, r2_score

mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"MAE = {mae:.2f}, R2 = {r2:.3f}")

# Coefficients tell you direction & strength
for col, coef in zip(X.columns, pipeline.named_steps["model"].coef_):
    print(f"{col}: {coef:.2f}")
```

Linearity assumptions: linear regression assumes a roughly linear relationship, independent errors, and no collinear features. Check residual plots; if they show a pattern, consider polynomial features or a non-linear model.

Logistic Regression (Classification)

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

y = df["is_churned"]
X = df[["spend", "support_tickets", "tenure"]]

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000, class_weight="balanced"))
])
pipe.fit(X_train, y_train)

y_pred = pipe.predict(X_test)
y_prob = pipe.predict_proba(X_test)[:, 1]  # P(churn)

print(confusion_matrix(y_test, y_pred))
# [[TN FP]
#  [FN TP]]

```

Decision Trees & Random Forests

```

from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from sklearn.ensemble import GradientBoostingRegressor

rf = RandomForestRegressor(n_estimators=300, random_state=42,
                           max_depth=10, min_samples_leaf=5)
rf.fit(X_train, y_train)

# Feature importance (built-in)
for col, imp in sorted(zip(X.columns, rf.feature_importances_),
                       key=lambda t: -t[1]):
    print(f"{col}: {imp:.3f}")

```

Random forests reduce overfitting by averaging many trees trained on bootstrap samples. Gradient boosting (XGBoost, LightGBM) trains trees sequentially, correcting previous errors — usually the strongest tabular model.

Hyperparameter tuning: use [GridSearchCV](#) or [RandomizedSearchCV](#) with cross-validation to find good hyperparameters. Search over a sensible grid, not every combination:

```

from sklearn.model_selection import RandomizedSearchCV
param_grid = {
    "model__n_estimators": [200, 400],
    "model__max_depth": [5, 10, None],
    "model__min_samples_leaf": [2, 5]
}
search = RandomizedSearchCV(rf, param_grid, cv=5, n_iter=20,
                            scoring="neg_mean_absolute_error", random_state=42)
search.fit(X_train, y_train)
print(search.best_params_)

```

XGBoost (the extra step)


```
# pip install xgboost
from xgboost import XGBRegressor

xgb = XGBRegressor(n_estimators=500, learning_rate=0.05, max_depth=6,
                   subsample=0.8, colsample_bytree=0.8, random_state=42)
xgb.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)
```

Model Comparison Discipline

1. Start with a simple baseline (mean prediction, or a linear model).
2. Add complexity only when it beats the baseline on *validation* data.
3. Compare models with identical cross-validation folds.
4. Report the metric on the untouched test set once, at the end.
5. Check residuals and the worst cases — global metrics hide systematic failures.

Explaining Predictions with SHAP

```
# pip install shap
import shap

explainer = shap.TreeExplainer(xgb)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
```

SHAP tells you *which features pushed each prediction up or down* — essential for trusting and defending your model.

Practice Task: On your dataset, train a linear model, a random forest, and an XGBoost model (regression or classification). Compare them with the same 5-fold cross-validation, tune the best one with `RandomizedSearchCV`, report final metrics on the held-out test set, and produce a SHAP summary plot. Write a short "model card" describing the winner, its metrics, and its top three features.

12. Clustering & Dimensionality Reduction

CHAPTER 12

Unsupervised Learning

Without labels, we look for **structure**: groups of similar customers, low-dimensional views of high-dimensional data, or anomalies. Clustering and dimensionality reduction are the two workhorses.

K-Means Clustering

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X = StandardScaler().fit_transform(df[["spend", "tenure", "support_tickets"]])

model = KMeans(n_clusters=4, n_init=10, random_state=42)
df["cluster"] = model.fit_predict(X)

# Profile the segments
print(df.groupby("cluster")[["spend", "tenure", "support_tickets"]].mean())
```

Scale before clustering: k-means minimizes Euclidean distance, so a variable measured in rupees dominates one measured in years. Always standardize continuous features first.

Choosing the Number of Clusters

k-means needs k up front. Heuristics:

- **Elbow method:** plot inertia vs k; pick the "elbow" where gains flatten.
- **Silhouette score:** measure how tight clusters are vs how far apart they are; higher is better.
- **Domain sense:** clusters must map to actionable segments — 4 clear business segments beat 12 statistically optimal ones.

```
from sklearn.metrics import silhouette_score

for k in range(2, 9):
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    labels = km.fit_predict(X)
    print(k, round(silhouette_score(X, labels), 3))
```

Hierarchical Clustering

Agglomerative clustering builds a tree (dendrogram) of nested clusters. It needs no k in advance and gives interpretable structure — great for taxonomies and small-to-medium data.

```
from scipy.cluster.hierarchy import linkage, fcluster, dendrogram
import matplotlib.pyplot as plt

Z = linkage(X, method="ward")
dendrogram(Z, truncate_mode="level", p=5)
plt.title("Dendrogram")
plt.show()
```

DBSCAN (Density-Based)

DBSCAN finds clusters as dense regions separated by sparse ones. It does *not* force every point into a cluster — points in low-density areas become noise. That makes it ideal when data has outliers or arbitrary shapes.

```
from sklearn.cluster import DBSCAN

db = DBSCAN(eps=0.5, min_samples=5)
labels = db.fit_predict(X) # -1 means "noise"
df["cluster"] = labels
print(df["cluster"].value_counts())
```

Dimensionality Reduction: PCA

Principal Component Analysis projects data onto the directions of greatest variance, reducing the number of columns while keeping most of the information.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X2d = pca.fit_transform(X)
print(pca.explained_variance_ratio_.sum()) # variance kept by 2 components
```

Uses: denoising, visualization of high-dimensional data, and speed. **Caution:** principal components are combinations of features and are not directly interpretable in business terms.

Embeddings & Modern Representations

Beyond PCA, modern tools learn dense vector representations:

- **Text embeddings** (e.g., from sentence-transformers) map words/sentences to vectors so similarity becomes distance — used for search, clustering documents, and topic discovery.
- **t-SNE / UMAP** produce striking 2D visualizations of high-dimensional structure (great for exploration, less good for clustering decisions).

```
# UMAP for exploration visualization
# pip install umap-learn
import umap
embedding = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=42).fit_transform(X)
```

Segment Validation

Clusters are only useful if they *differ in ways you can act on*. Validate with:

- Distribution plots comparing clusters on key metrics.
- Stability: re-run with different seeds/params — do the segments persist?
- Business check: can you name each cluster and suggest a strategy for it?

Practice Task: Cluster your customers (or another natural unit) with k-means. Choose k with the elbow and silhouette methods, profile each segment, and give each a memorable business name and a one-line strategy. Then use DBSCAN on the same data and compare which points are treated as noise. Finally, visualize the segments with PCA or UMAP.

13. Time Series Analysis

CHAPTER 13

What Makes Time Series Special

Time series data has **order**: rows are not exchangeable. Neighboring observations are correlated, so the tools from Chapter 10 (which assume independence) do not apply directly. Time series work powers forecasting — demand, revenue, load, prices — and anomaly detection.

Components of a Time Series

- **Trend** — long-term direction (up, down, flat).
- **Seasonality** — repeating pattern with fixed period (daily, weekly, yearly, holiday effects).
- **Cyclicity** — longer, irregular cycles (economic booms/busts).
- **Noise** — irregular, unpredictable movement.

Decompose any series with `statsmodels` :

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

df = df.set_index("date").sort_index()
result = seasonal_decompose(df["revenue"], model="additive", period=12)
result.plot()
plt.show()
```

Stationarity

Most classical models assume **stationarity**: constant mean and variance over time (no trend, no changing seasonality).

```
from statsmodels.tsa.stattools import adfuller

stat, p_value = adfuller(df["revenue"])[1:2]
print(p_value) # p < 0.05 -> stationary
```

Non-stationary? Common fixes:

- Differencing: `df["revenue"].diff()` removes trend; seasonal differencing removes seasonality.
- Log transform stabilizes variance.
- Model the components explicitly instead (Prophet, or include trend/season dummies).

Autocorrelation

Check how a series correlates with its own past. **ACF** shows correlation at each lag; **PACF** shows the partial correlation after removing intermediate lags. These plots guide ARIMA orders.

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

plot_acf(df["revenue"].diff().dropna(), lags=24)
plot_pacf(df["revenue"].diff().dropna(), lags=24)
plt.show()
```

ARIMA

ARIMA combines **AR** (autoregression on past values), **I** (differencing to reach stationarity), and **MA** (moving average of past errors): `ARIMA(p, d, q)`.

```
from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(df["revenue"], order=(1, 1, 1))
fit = model.fit()
print(fit.summary())

# Forecast the next 12 periods
forecast = fit.forecast(steps=12)
print(forecast)
```

Choosing p, d, q: use `auto_arima` (pmdarima) or ACF/PACF plots plus AIC comparison of a small grid.

Never train/test randomly on time series. Split chronologically: train on the past, test on the future (e.g., last 20% of dates). A random split leaks the future into training and flatters your scores.

Holidays & Exogenous Variables

Real demand is driven by calendars (Diwali, Christmas, fiscal year-end). Two ways to handle it:

- Add holiday/external regressors (weather, promotions, events) to a model that supports them.
- Use a model designed for this, such as **Prophet** or SARIMAX with exogenous regressors.

```
# Prophet: trend + seasonality + holidays handled automatically
# pip install prophet
from prophet import Prophet

forecast_df = df.reset_index().rename(columns={"date": "ds", "revenue": "y"})
m = Prophet(seasonality_mode="multiplicative",
            yearly_seasonality=True, weekly_seasonality=True)
m.add_country_holidays(country_name="IN")
m.fit(forecast_df)
future = m.make_future_dataframe(periods=90)
forecast = m.predict(future)
m.plot(forecast)
```

Evaluating Forecasts

Report error relative to the scale of the data:

- **MAE** — average error in units.
- **MAPE** — average percentage error (be careful with near-zero values).
- **RMSE** — penalizes large errors.
- **MASE** — compares against a naive baseline; < 1 means better than "predict last value."

```
from sklearn.metrics import mean_absolute_error

y_true, y_pred = test["revenue"], test["forecast"]
print("MAE:", mean_absolute_error(y_true, y_pred))
```

When to Use What

Situation	Tool
Simple trend/seasonality, business forecasting	Prophet, SARIMA
Many series with shared patterns	Global models (LightGBM with features, deep learning)
High-frequency (seconds-minutes)	Exponential smoothing, ML with rolling features
Exogenous drivers matter (weather, price)	SARIMAX, Prophet regressors, ML

Practice Task: Find a monthly or daily public time series (sales, weather, stock, energy). Decompose it, test for stationarity, and build an ARIMA (or Prophet if holidays are relevant) forecast on a chronological train/test split. Plot the forecast with confidence bands and report MAE/MAPE on the test period.

14. NLP & Text Analytics

CHAPTER 14

Text as Data

Reviews, support tickets, emails, social media, and documents are rich sources of signal. **Natural language processing (NLP)** turns unstructured text into numbers a model can use. Typical goals: sentiment, topic discovery, classification, search, and summarization.

The NLP Pipeline

```
import re
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer

docs = df["review_text"].tolist()

# 1. Clean
def clean(t):
    t = t.lower()
    t = re.sub(r"http\S+", "", t)      # remove URLs
    t = re.sub(r"^[a-z0-9\s]", "", t)  # keep letters and digits
    return re.sub(r"\s+", " ", t).strip()

df["clean"] = df["review_text"].map(clean)

# 2. Tokenize + count/TF-IDF
vectorizer = TfidfVectorizer(max_features=2000, stop_words="english")
X = vectorizer.fit_transform(df["clean"])
print(X.shape)  # (n_docs, 2000)
```

TF-IDF in Practice

TF-IDF weights words by how often they appear in a document (term frequency) and how rarely they appear across documents (inverse document frequency). Common words like "the" get near-zero weight; distinctive words carry the signal.

```
# Which terms are most distinctive overall?
names = vectorizer.get_feature_names_out()
top = X.sum(axis=0).A1
for idx in top.argsort()[-15:]:
    print(names[idx], round(top[idx], 2))
```

When TF-IDF fails: for short texts (tweets, chat) and for capturing word order ("not bad" vs "bad"), count/TF-IDF is too weak. Use embeddings (next section) or n-grams (`ngram_range=(1,2)`) instead.

Modern Embeddings

Embeddings map words or sentences to dense vectors where similar meanings are near each other. Sentence embeddings turn any text into a fixed-size vector, so you can cluster documents, build search, or feed the vector into any ML model.

```
# pip install sentence-transformers
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
vectors = model.encode(df["clean"].tolist()) # shape (n, 384)
```

Sentiment Analysis

Two paths: a **lexicon** (a word → sentiment score dictionary, fast but crude) or a **trained classifier** (accurate but needs labeled data). With no labels, start with a pretrained model:

```
# pip install transformers
from transformers import pipeline

sentiment = pipeline("sentiment-analysis")
results = sentiment(df["clean"].tolist()[:100])
```

Topic Modeling

Topic modeling discovers themes in a corpus without labels. **LDA** is the classic method:

```
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer

count = CountVectorizer(max_features=3000, stop_words="english")
Xc = count.fit_transform(df["clean"])

lda = LatentDirichletAllocation(n_components=5, random_state=42)
lda.fit(Xc)

for topic_idx, topic in enumerate(lda.components_):
    words = [count.get_feature_names_out()[i] for i in topic.argsort()[-8:]]
    print(f"Topic {topic_idx}: {' '.join(words)}")
```

Classification of Text

Once text is vectors, supervised learning proceeds exactly as in Chapter 11:

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X = vectorizer.fit_transform(df["clean"])
y = df["category"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)
print(clf.score(X_test, y_test))
```

Ethical Care with Text

- Text data often contains **personal information** — names, emails, locations. Anonymize before sharing or storing.
- Models learn the biases of the text they train on (Chapter 17).
- Sentiment and topic models can be **wrong in high-stakes ways**; never let them make decisions about people without review.

Practice Task: Take a public text dataset (e.g., customer reviews or news headlines). Clean and vectorize it, compute a sentiment score per document, cluster the documents with embeddings, and run topic modeling to name 4-6 themes. Write a summary of the top three themes and how the sentiment varies across them.

15. Feature Engineering & Model Deployment

CHAPTER 15

Features Decide Performance

Better features usually beat better algorithms. **Feature engineering** is creating columns that make the pattern easier for the model to see: dates → weekday/holiday, IDs → counts/frequencies, free text → sentiment, raw numbers → ratios and interactions.

A Feature Engineering Toolkit

```
# Dates -> calendar features
df["weekday"] = df["date"].dt.dayofweek
df["hour"] = df["date"].dt.hour
df["is_weekend"] = df["weekday"] >= 5
df["month"] = df["date"].dt.month

# Counts from IDs (frequency encoding)
df["city_freq"] = df["city"].map(df["city"].value_counts())

# Ratios & interactions
df["spend_per_ticket"] = df["spend"] / (df["support_tickets"] + 1)
df["spend_per_month"] = df["spend"] / (df["tenure"] + 1)

# Aggregates (rolling features for time series)
df["spend_7d_avg"] = df.groupby("customer_id")["spend"].transform(lambda s: s.rolling(7).mean())
```

Target leakage in features: never create a feature from information that wouldn't be available at prediction time — e.g., "spend_next_month" as a feature, or a rolling average computed over the future. Leaky features make great offline scores and broken production systems.

sklearn Pipelines & ColumnTransformer

Combine preprocessing and modeling into one artifact that cannot leak:

```

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor

num_cols = ["age", "spend", "support_tickets"]
cat_cols = ["city", "plan"]

pre = ColumnTransformer([
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ]), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
])

model = Pipeline([
    ("pre", pre),
    ("rf", RandomForestRegressor(n_estimators=300, random_state=42))
])
model.fit(X_train, y_train)

```

This whole object can be saved with `joblib` and reused in production unchanged:

```

import joblib
joblib.dump(model, "model/spend_model.joblib")
# later / elsewhere:
loaded = joblib.load("model/spend_model.joblib")
prediction = loaded.predict(new_data)

```

Feature Stores

In a real product, the same features are needed for **training** and for **live scoring**. A **feature store** (Feast, Databricks Feature Store, BigQuery ML features) is the shared catalog that serves both, so online and offline features never drift apart.

Problem	Feature store answer
Same feature computed differently in train vs serve	One definition, served everywhere
Stale features in production	Online/offline consistency checks
Teams re-inventing the same features	Shared, documented catalog with owners

Model Serving Options

Approach	Use for	Example
Batch scoring	Scores computed nightly/on schedule	Cron job → warehouse table
REST API	Real-time predictions behind an app	FastAPI + joblib model
Serverless function	Event-driven, low traffic	Cloud Function / Lambda
Managed serving	Scaling, monitoring, auto-ML ops	Vertex AI, SageMaker, Azure ML

```
# A minimal real-time serving API (FastAPI)
# pip install fastapi uvicorn joblib
from fastapi import FastAPI
from pydantic import BaseModel
import joblib, pandas as pd

app = FastAPI()
model = joblib.load("model/spend_model.joblib")

class Input(BaseModel):
    age: float
    spend: float
    support_tickets: float
    city: str
    plan: str

@app.post("/predict")
def predict(x: Input):
    row = pd.DataFrame([x.model_dump()])
    return {"predicted_spend": float(model.predict(row)[0])}
```

Monitoring in Production

Deploying is the beginning, not the end. Monitor:

- **Data drift** — the input distribution changes over time (compare live inputs to training data).
- **Concept drift** — the relationship you learned stops holding (retrain schedules, performance probes).
- **Performance** — when ground truth arrives late, track it over time.
- **Operational health** — latency, error rates, and staleness of features.

Tools: **Evidently** (data drift), **Prometheus/Grafana** (operations), simple dashboards over logged predictions.

Practice Task: Take your best model from Chapter 11, wrap its preprocessing + model in a single `Pipeline` with a `ColumnTransformer`, save it with `joblib`, and expose it through a minimal FastAPI endpoint. Test it with sample JSON. Then write a short monitoring plan: which columns would you watch for drift and how would you know the model needs retraining?

16. Data Science at Scale & in the Cloud

CHAPTER 16

When Local Python Stops Being Enough

pandas on a laptop handles millions of rows. Beyond that — terabytes, distributed compute, collaboration, scheduled pipelines — you move to the cloud and to distributed engines. The skills transfer: you still write SQL and Python; the platform changes.

The Cloud Data Stack

Layer	Google Cloud	AWS	Azure
Warehouse/analytics	BigQuery	Redshift	Azure Synapse / Fabric
Object storage	GCS	S3	Azure Blob
Notebooks	Colab / Vertex Workbench	SageMaker Studio	Azure ML
Managed ML	Vertex AI	SageMaker	Azure ML
Distributed compute	Dataproc (Spark)	EMR	HDInsight

BigQuery for Data Science

BigQuery answers SQL on huge tables almost instantly. It also trains models and makes predictions without moving data:

```
-- Train a model in SQL (BigQuery ML)
CREATE OR REPLACE MODEL `project.dataset.churn_model`
OPTIONS(model_type='LOGISTIC_REG', input_label_cols=['is_churned']) AS
SELECT spend, support_tickets, tenure, is_churned
FROM `project.dataset.customer_features`;

-- Predict
SELECT *
FROM ML.PREDICT(MODEL `project.dataset.churn_model`,
                (SELECT spend, support_tickets, tenure FROM `project.dataset.new_customers`));
```

Read this before computing: cloud services bill by bytes scanned and compute time. Filter early, select only needed columns, partition and cluster tables, and use previews before running queries. A `SELECT *` over a huge table is an expensive habit.

Spark: Distributed Python

Apache Spark runs pandas-style operations across a cluster. Its DataFrame API feels familiar:

```
# pip install pyspark
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("ds").getOrCreate()

df = spark.read.parquet("gs://my-bucket/events/")
df.printSchema()

from pyspark.sql import functions as F
summary = (df
    .filter(F.col("event_type") == "purchase")
    .groupBy("customer_id")
    .agg(F.sum("amount").alias("total_spend"),
        F.count("*").alias("orders"))
    .orderBy(F.col("total_spend").desc())
    .limit(1000))
summary.show()
```

Spark **lazily** builds a plan and only executes when an action (`show` , `count` , `write`) runs — so you can chain transforms without materializing intermediates.

Scaling Machine Learning

Scikit-learn does not scale horizontally. Options:

- **Spark MLlib** — distributed training of classical models.
- **XGBoost/LightGBM + Dask** — distributed gradient boosting.
- **BigQuery ML** — train in-warehouse without a cluster.
- **Deep learning frameworks** (PyTorch/TensorFlow) — GPU training, distributed data parallel.

MLOps: The Production Discipline

MLOps applies DevOps practices to machine learning so models ship reliably and stay healthy:

- **Versioning** — code, data, and models all versioned (`dvc` or cloud ML metadata).
- **Reproducible runs** — same inputs → same outputs; log parameters and metrics (MLflow).
- **CI/CD** — tests run before a model is promoted; automated pipelines rebuild on data changes.
- **Registry** — model registry holds approved versions with metadata (Vertex AI Model Registry, MLflow).
- **Monitoring & retraining** — drift detection and scheduled retraining (Chapter 15).

A Pragmatic Cloud Workflow

1. Prototype locally in a notebook on a sample.
2. Move heavy ETL to the warehouse (BigQuery SQL) or a Spark job.
3. Train with managed ML (or BigQuery ML) so jobs are tracked and reproducible.
4. Register the model and serve via batch or an endpoint.
5. Schedule pipelines and monitor drift.

Practice Task: Take your cleaned dataset and load it into BigQuery's sandbox (free). Reproduce your EDA chapter as SQL: distribution stats, a grouped comparison, and a correlation. If you feel ambitious, train a BigQuery ML model with `CREATE MODEL` and compare its metrics with your scikit-learn model.

17. Ethics, Bias & Responsible AI

CHAPTER 17

Why Ethics is a Technical Skill

Models are not neutral. They are trained on the world as it is — including its biases — and then applied at scale, amplifying both strengths and harms. Responsible data science treats fairness, privacy, transparency, and accountability as **engineering requirements**, not afterthoughts.

Where Bias Enters

- **In the data:** historical discrimination, biased sampling, small or missing groups, proxy variables (e.g., postal code standing in for race).
- **In the label:** the target itself may encode bias — e.g., "employee performance" scores influenced by the boss's bias.
- **In the model:** optimization can sharpen disparities even when the goal seems neutral.
- **In deployment:** who uses the model, on whom, with what recourse, and how errors fall.

Proxy problem: even if you remove a protected attribute (race, gender), correlated features like income, address, or name can reintroduce it. Removing the column is not removing the bias — audit the model's behavior across groups, not just the inputs.

Fairness Metrics

There is no single "fairness." Different definitions can conflict, so you must choose and defend one per context:

Metric	Question it answers
Demographic parity	Are positive outcomes equally likely across groups?
Equalized odds	Are error rates equal across groups (given true label)?
Equal opportunity	Is recall equal across groups?
Calibration	Do predicted probabilities mean the same across groups?

```
# Example: compare recall across groups
for group in df["group"].unique():
    mask = (df["group"] == group) & (df["y_true"] == 1)
    recall = df.loc[mask, "y_pred"].mean()
    print(group, round(recall, 3))
```

Privacy & Data Protection

- **Minimize:** collect and keep only what is needed.

- **Anonymize:** remove identifiers; beware that combining datasets can re-identify people.
- **Consent & purpose:** use data for what people were told it would be used for.
- **Security:** encrypt at rest and in transit, restrict access, and never store secrets in code.
- **Regulations:** know the rules that apply to you — GDPR (EU), CCPA (California), and India's DPDP Act 2023.

Transparency & Explainability

Stakeholders and regulators need to understand what a model does and why it reached a decision.

Techniques:

- **SHAP / permutation importance** — which features drove each prediction (Chapter 11).
- **Model cards** — a one-page spec: purpose, data, limitations, evaluation, intended and prohibited uses.
- **Documentation of data lineage** — where data came from, how it was cleaned (Chapter 3).
- **Human review for high-stakes** — hiring, lending, and healthcare decisions need a human in the loop with clear recourse.

Accountability & Governance

Someone must own the model's behavior. Good governance looks like:

- A named owner for every model in production.
- Review gates before deployment (ethics + technical sign-off).
- Incident response for harmful outputs.
- Monitoring bias metrics after launch, not just accuracy (Chapters 15 and 16).
- Clear paths to challenge or override a model's decision.

Practical Checklist Before Shipping

1. List the people affected and the stakes if the model is wrong.
2. Check representation and performance across groups.
3. Document data provenance and limitations.
4. Define who is accountable and how decisions can be challenged.
5. Plan monitoring for drift, bias, and performance.
6. Write the model card.

Practice Task: For your best model, write a model card with: purpose, training data, evaluation metrics (overall and per group), known limitations, intended uses, and prohibited uses. Then audit the model across at least two subgroups of your data (e.g., by city or plan) and report at least one fairness metric per group. State explicitly what your model should NOT be used for.

18. Projects, Portfolio & Career Path

CHAPTER 18

The Portfolio Is the Resume

Employers cannot see your coursework, but they can read your code and notebooks. A small number of **complete, documented projects** beats a long list of started-but-unfinished ones. Each project should tell a story: problem → data → analysis → model → conclusion → action.

The Three Signature Projects

Project	What it proves	Builds on
1. End-to-end EDA + dashboard	Wrangling, analysis, visualization, communication	Ch. 3–6
2. Predictive model with deployment	ML workflow, pipelines, serving, monitoring	Ch. 10–15
3. A specialization (time series / NLP / clustering)	Depth in one area	Ch. 12–14

What Makes a Project Stand Out

- **A real question**, not a toy dataset demo — "predict my city's bike-share demand," "segment our store's customers."
- **Clean code**: `README.md` with setup instructions, well-named modules, no notebooks full of dead cells.
- **Good hygiene**: `requirements.txt`, `.gitignore`, a license, and commits with meaningful messages.
- **A model card** (Chapter 17) and an honest discussion of limitations.
- **Deployment**: a live Streamlit app or FastAPI endpoint is worth more than ten notebooks.
- **Visuals**: a strong readme image or dashboard screenshot.

Portfolio Checklist

1. GitHub profile with 2-3 pinned repositories, each documented.
2. A one-page resume: skills, 2-3 projects with measurable outcomes, tools.
3. A LinkedIn presence with your projects linked.
4. Optional but powerful: a short blog post or notebook walkthrough for your best project.

Interview Preparation

Typical data science interviews have several rounds:

Round	Focus	How to prepare
Take-home / case study	End-to-end thinking on a business problem	Practice scoping: metrics, data needs, risks
SQL	Joins, aggregations, window functions	Daily drills on BigQuery sample data
Statistics & ML theory	Hypothesis testing, bias-variance, metrics	Explain every concept you used in your projects
Python/coding	Data structures and pandas/scikit-learn	LeetCode easy-medium + pandas exercises
Behavioral	Communication, judgment, teamwork	STAR stories from your projects

Career Roadmap

- 1. Foundations (weeks 1-8):** Chapters 1–9 — Python, pandas, EDA, stats, SQL.
- 2. Modeling (weeks 9-14):** Chapters 10–15 — ML, pipelines, deployment.
- 3. Depth (weeks 15-18):** Pick a track — forecasting, NLP, or cloud/MLOps (Chapters 13, 14, 16).
- 4. Portfolio (weeks 19-24):** Build and polish the three signature projects; write model cards.
- 5. Job search:** apply widely, ask for feedback on rejections, and keep one project improving while you interview.

Growth mindset: data science changes fast. The durable skills are the ones this book emphasizes — asking the right question, clean data work, honest statistics, clear communication, and responsible practice. Tools change; those skills compound.

Community & Continued Learning

- Kaggle competitions (start with getting top 25%, not winning).
- Open-source: contribute documentation or bug fixes to the libraries you use.
- Meetups and conferences (local or virtual) to meet practitioners.
- Follow the maintainers and papers of your favorite tools.

Practice Task: Write the README for your best project today — problem, data, method, results, limitations, and how to run it. Add a screenshot of your best chart or dashboard. Then list the two weaknesses in your current portfolio and the specific project that will fix each one.

19. References & Resources

CHAPTER 19

Glossary

Term	Meaning
Baseline	A simple reference model (mean prediction, majority class) every better model must beat.
Bias (statistical)	Systematic error from a model being too simple (underfitting).
Bias (fairness)	Systematic disadvantage of one group caused by data, labels, model, or deployment.
Bootstrap	Resampling with replacement to estimate uncertainty without formulas.
Confidence interval	Range of plausible values for a parameter from a procedure with known coverage.
Cross-validation	Repeated train/validate splits that estimate generalization performance.
Data drift	The input distribution shifts from what the model was trained on.
ELT	Extract, Load, Transform — load raw data first, transform in the warehouse.
Embedding	A dense numeric vector representing text (or other objects) so similarity is distance.
Exploratory Data Analysis	The disciplined exploration of data to find patterns, problems, and hypotheses.
Feature	An input column a model uses to make predictions.
Feature store	A system serving the same features to training and live scoring.
Imputation	Filling missing values with estimates (mean, median, mode, model prediction).
Leakage	Information from the future/test set entering training, inflating performance.
Model card	A one-page spec of a model: purpose, data, metrics, limitations, uses.
Overfitting	Memorizing training noise; great train scores, poor test scores.
p-value	Probability of the observed result (or more extreme) if the null hypothesis were true.
Pipeline	Chained preprocessing + model steps executed and saved as one object.
Power	The probability of detecting a true effect ($1 - \text{Type II error}$).
Stationarity	A time series with constant mean and variance over time.
Train/test split	Holding out part of the data to evaluate generalization.
Variance (statistical)	Error from a model being too sensitive to training data (overfitting).
Window function	A SQL function computing over a row set without collapsing rows.

Cheatsheet: Python Data Science

```
# pandas
df = pd.read_csv("file.csv")
df.groupby("col")["value"].agg(["mean", "count"])

# statistics
from scipy import stats
stats.ttest_ind(a, b)
stats.pearsonr(x, y)

# sklearn
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.ensemble import RandomForestRegressor

# visualize
import matplotlib.pyplot as plt
import seaborn as sns
sns.histplot(data, kde=True)
```

Cheatsheet: SQL

```
SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
LIMIT

-- window
SUM(x) OVER (PARTITION BY g ORDER BY t)
ROW_NUMBER() OVER (PARTITION BY g ORDER BY t)
LAG(x) OVER (PARTITION BY g ORDER BY t)

-- CTE
WITH t AS (SELECT ...) SELECT ... FROM t;
```

Public Datasets to Practice On

- **Kaggle** — competitions and datasets with notebooks to learn from.
- **UCI ML Repository** — classic datasets for every task.
- **Google BigQuery public datasets** — huge real-world tables for SQL and cloud practice.
- **Statsmodels / seaborn sample datasets** — built-in practice data.
- **Awesome Public Datasets** — a curated index of everything else.

Books & Documentation to Keep Around

- **pandas documentation** — the reference you will use daily.
- **scikit-learn user guide** — best in-class explanations of every algorithm.
- **"Python for Data Analysis"** (Wes McKinney) — pandas depth.

- **"An Introduction to Statistical Learning"** (James, Witten, Hastie, Tibshirani) — the theory behind Chapters 10–12.
- **"Storytelling with Data"** (Cole Nussbaumer Knaflic) — chart design and narrative.
- **"The Elements of Statistical Learning"** (Hastie et al.) — the deeper graduate-level companion.

Concepts to Revisit Per Chapter

Chapter	Key ideas to remember
1	Data science = stats + code + domain. Frame the problem first.
2	Virtual environments, Git, notebooks for exploration, modules for production.
3	Document provenance and licenses; respect APIs and scraping rules.
4	Missingness strategy, dedupe with intent, investigate outliers, never leak.
5	EDA generates testable hypotheses; mean vs median; correlation $\neq$ causation.
6	Pick the right chart, label everything, keep axes honest.
7	Base rates matter (Bayes); CLT powers inference; CI is a procedure property.
8	Pre-register tests, size for power, don't peek, correct for multiple comparisons.
9	Execution order, joins and cardinality, window functions, CTEs.
10	Generalize, not memorize; leakage; bias-variance; metrics match the task.
11	Pipeline everything; baseline first; SHAP to explain.
12	Scale features before clustering; validate segments by business actionability.
13	Chronological splits; decompose; stationarity; forecast error vs baseline.
14	TF-IDF for medium texts; embeddings for meaning; anonymize text data.
15	Features beat algorithms; no future leaks; monitor drift after launch.
16	Filter early on the cloud; MLflow for runs; register models.
17	Audit across groups; write model cards; keep a human for high stakes.
18	Complete documented projects beat many demos; interviews test fundamentals.

Final Task: Return to the question you wrote in Chapter 1's exercise. By now you should have the data, the analysis, the model, and the narrative for it. Write the final README, add the model card, deploy the dashboard or API, and share it. Congratulations — you are a data scientist with a portfolio.

2. Setting Up Your Data Science Environment

CHAPTER 2

Why Environment Setup Matters

A reproducible environment is the foundation of every data science project. If your code works on your machine but not a teammate's, it is not yet science. This chapter sets up a clean, version-controlled Python workspace you will use for the rest of the book.

Install Python

Install a recent Python 3 (3.10 or newer) from python.org. On Windows, tick **"Add Python to PATH"** during installation. Verify:

```
python --version
# Python 3.12.x
```

Python vs Anaconda: Anaconda bundles Python with a large scientific stack and is convenient for beginners. For this book a plain Python install plus [pip](#) and virtual environments is cleaner and matches how professionals work. Choose Anaconda only if you prefer its ecosystem.

Virtual Environments

A **virtual environment** isolates the packages of one project from the rest of your system, so different projects can use different versions of the same library without conflicts.

```
# Create a project and its environment
mkdir data-science-project
cd data-science-project
python -m venv .venv

# Activate (Windows)
.venv\Scripts\activate
# Activate (macOS / Linux)
source .venv/bin/activate

# Install the scientific stack
pip install numpy pandas matplotlib seaborn scikit-learn jupyter scipy statsmodels plotly
```

Record your dependencies so the project is reproducible:

```
pip freeze > requirements.txt
# or, the modern alternative
pip install pip-tools
```


When to upgrade: data science libraries release frequently. Pin exact versions in `requirements.txt` for reproducibility, but test upgrades in a branch so you know what changes break your work.

Jupyter Notebooks

Jupyter notebooks combine code, output, text, and visualizations in one document — ideal for exploration. Launch from your activated environment:

```
jupyter notebook
# or the newer web app
jupyter lab
```

Notebooks are **great for exploration** but **poor for production** code. A healthy pattern: explore in notebooks, then move the working logic into plain `.py` modules that notebooks import.

Notebook hygiene: notebooks silently keep stale state (a variable from a deleted cell still exists). Run cells top-to-bottom and occasionally restart the kernel and run all to confirm the notebook actually works.

VS Code as Your Editor

VS Code is a great all-rounder for data science: full Python debugging, Git integration, and Jupyter support built in.

- Install the **Python** and **Jupyter** extensions.
- Open your project folder and select your `.venv` as the interpreter (`Ctrl+Shift+P` → *Python: Select Interpreter*).
- Use the built-in terminal so your active environment is consistent with the editor.

Git & GitHub

Version control is non-negotiable: it is your undo history, your backup, and your portfolio.

```
git init
git add .
git commit -m "Initial data science project"
git branch -M main
git remote add origin https://github.com/you/data-science-project.git
git push -u origin main
```

Add a `.gitignore` to avoid committing junk:

```
# .gitignore
.venv/
__pycache__/
.ipynb_checkpoints/
.DS_Store
*.ipynb # optional – some teams commit notebooks deliberately
```

Reproducibility gold standard: environments can also be captured as `conda env export > environment.yml` or as a `Dockerfile`. For this book, `requirements.txt` + a README with setup instructions is enough.

Your First Data Science Script

```
import numpy as np
import pandas as pd

data = pd.DataFrame({
    "age": [25, 31, 45, 28, 52, 33],
    "spend": [120, 250, 480, 180, 610, 220]
})

print(data.describe())
#      age      spend
# count  6.0    6.000000
# mean   35.7   310.000000
# ...
```

Practice Task: Create a folder `data-science-project`, build the virtual environment, install the scientific stack, initialize Git, and push a notebook that loads any CSV from the internet (search "example CSV datasets") and prints its shape and first rows.

3. Data Acquisition & Collection

CHAPTER 3

Where Data Comes From

Every project starts with getting data into a form you can analyze. Common sources:

Source	Examples	Typical Format
Flat files	CSV, Excel, JSON, Parquet, log files	Files on disk or in cloud storage
Databases	PostgreSQL, MySQL, SQLite, warehouses	SQL queries
APIs	REST/GraphQL endpoints, public datasets	JSON/XML over HTTP
Web	Public pages, review sites	HTML (scraped)
Streaming	Clickstreams, IoT, financial feeds	Event streams (Kafka, WebSockets)
Surveys/experiments	Questionnaires, lab logs	Structured tables

Reading Flat Files with pandas

```
import pandas as pd

# CSV
df = pd.read_csv("data/sales.csv")

# Excel (needs openpyxl: pip install openpyxl)
df = pd.read_excel("data/sales.xlsx", sheet_name="2025")

# JSON
df = pd.read_json("data/events.json")

# Parquet (fast columnar, pip install pyarrow)
df = pd.read_parquet("data/events.parquet")

# Inspect what you got
print(df.shape)      # (rows, columns)
print(df.head())
print(df.dtypes)
```

Always inspect before assuming: print `df.shape` and `df.dtypes` immediately. Files often contain extra header rows, encoding surprises, or unexpected delimiters. Use `sep=";"`, `encoding="utf-8-sig"`, or `skiprows=2` as needed.

Querying Databases

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("data/app.db")
df = pd.read_sql_query("SELECT * FROM orders WHERE created_at > '2025-01-01'", conn)
conn.close()
```

For PostgreSQL and other servers, use connection strings and a driver such as `psycopg2` or `SQLAlchemy`. SQL itself is covered in depth in Chapter 9 — this chapter focuses on getting the data out.

Never put credentials in code. Use environment variables or a secrets manager:

```
import os
DATABASE_URL = os.environ["DATABASE_URL"]
```

Fetching Data from APIs

Many services expose data over HTTP. The `requests` library makes calls easy:

```
import requests

resp = requests.get(
    "https://api.open-meteo.com/v1/forecast",
    params={"latitude": 21.17, "longitude": 72.83, "hourly": "temperature_2m"}
)
resp.raise_for_status()      # raise on HTTP errors
data = resp.json()           # parsed JSON
df = pd.DataFrame(data["hourly"])
```

Public dataset catalogs you can start with:

- **Kaggle Datasets** — kaggle.com/datasets (requires free account)
- **UCI Machine Learning Repository** — archive.ics.uci.edu
- **Google Dataset Search** — datasetsearch.research.google.com
- **Awesome Public Datasets** — github.com/awesomedata/awesome-public-datasets
- **Open-Meteo / OpenWeatherMap** — free weather APIs
- **Government open data** — e.g., data.gov.in, data.gov

Respect the API: read the terms of service, use authentication tokens when required, and be polite with rate limits (a `time.sleep(0.5)` between calls is common). Public APIs are shared infrastructure.

Web Scraping (Carefully)

When no API exists, scraping can collect public data:

```
from bs4 import BeautifulSoup
import requests

resp = requests.get("https://example.com/reviews")
soup = BeautifulSoup(resp.text, "html.parser")

titles = [el.text.strip() for el in soup.select("h2.review-title")]
print(titles[:5])
```

Scraping ethics: only scrape data you have permission to use. Check the site's terms, robots.txt, and the jurisdiction's data laws. Never scrape login-walled or personal data. For practice, prefer datasets or well-documented APIs.

Data Provenance & Documentation

Record where each dataset came from, when you downloaded it, its version, and its license. This matters for reproducibility and for legal/ethical use of the data.

```
# A tiny data source note you can keep beside your data
# Source: Open-Meteo archive (CC-BY-4.0)
# Downloaded: 2026-08-15
# Columns: time (ISO8601), temperature_2m (degC), humidity (percent)
```

Practice Task: Choose two sources from different categories (e.g., an API and a CSV dataset). Load both into pandas, save each to `data/raw/`, and write a short `sources.md` documenting provenance and license. Verify with `df.info()` that column types look sane.

4. Data Wrangling & Cleaning

CHAPTER 4

Why Cleaning Takes Most of Your Time

Real data is messy: missing values, inconsistent formats, duplicates, outliers, and impossible values. Practitioners commonly spend 60–80% of a project cleaning and preparing data. This chapter gives you a repeatable checklist.

The Wrangling Checklist

1. Look at the shape and types (`df.info()`).
2. Check for missing values (`df.isna().sum()`).
3. Remove or fix duplicates (`df.duplicated()`).
4. Standardize formats (dates, strings, categoricals).
5. Handle outliers and impossible values.
6. Engineer the columns you need (Ch. 15).

First Look

```
import pandas as pd
df = pd.read_csv("data/customers.csv")

df.info()           # columns, non-null counts, dtypes
df.isna().sum()     # missing values per column
df.nunique()        # distinct values per column
df.duplicated().sum() # duplicate rows
```

Handling Missing Values

Missingness matters: *why* a value is missing changes what you should do.

Type	Meaning	Common Strategy
Missing at random	Unrelated to the value itself	Drop rows or impute
Missing systematically	Related to the value (rich people don't answer income)	Add an "is missing" flag; model separately
Not applicable	Absence has meaning (no phone = no mobile)	Fill with a sentinel like <code>"none"</code>

```
# How much is missing
missing = df.isna().sum()
print(missing[missing > 0])

# Drop rows missing a critical column
df = df.dropna(subset=["customer_id"])

# Impute numeric columns with the median (robust to outliers)
for col in ["income", "age"]:
    df[col] = df[col].fillna(df[col].median())

# Impute categorical columns with the mode
df["city"] = df["city"].fillna(df["city"].mode()[0])
```

Do not impute blindly: imputation adds information you invented. For machine learning, a principled approach is to keep missingness and model it (a missing-value indicator column, or a model that handles `NaN` natively such as many tree models).

Duplicates

```
# Show duplicate rows
print(df[df.duplicated()])

# Drop exact duplicates
df = df.drop_duplicates()

# Drop duplicates on a key column, keeping the latest
df = df.sort_values("created_at").drop_duplicates(subset=["customer_id"], keep="last")
```

Duplicates are not always bad: in transactional data, two identical rows may be legitimate (two identical purchases). Only dedupe when the rows truly represent the same entity.

Types, Dates & Strings

```
# Convert to proper types
df["price"] = pd.to_numeric(df["price"], errors="coerce")
df["joined"] = pd.to_datetime(df["joined"], errors="coerce")

# Clean strings
df["city"] = (df["city"]
              .str.strip()           # trim whitespace
              .str.lower()          # normalize case
              .str.replace(r"\s+", "_", regex=True))

# Fix inconsistent categorical values
mapping = {"M": "Male", "F": "Female", "m": "Male", "f": "Female", "O": "Other"}
df["gender"] = df["gender"].map(mapping)

# Extract date parts
df["year"] = df["joined"].dt.year
df["weekday"] = df["joined"].dt.day_name()
```

Outliers & Impossible Values

First decide whether a value is *an error* or *an extreme but real observation*. A salary of ₹1,00,00,000 may be real; an age of 250 is impossible.

```
# Summary statistics expose impossibilities
print(df.describe())

# Flag impossible values
df.loc[df["age"] > 120, "age"] = pd.NA

# The IQR rule – a rough outlier screen (not a definition of "bad")
q1, q3 = df["spend"].quantile([0.25, 0.75])
iqr = q3 - q1
outside = (df["spend"] < q1 - 1.5 * iqr) | (df["spend"] > q3 + 1.5 * iqr)
print(df.loc[outside, "spend"])
```

Never silently delete outliers. An outlier often marks a data error, a segment worth studying, or a fraud signal. Investigate before you drop, and document the decision.

Normalization & Encoding

Numeric features on different scales distort distance-based algorithms and can slow training.

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# Standardize to mean 0, std 1 (default for many models)
scaler = StandardScaler()
df[["income", "age"]] = scaler.fit_transform(df[["income", "age"]])

# Min-max scale to [0,1]
df[["income", "age"]] = MinMaxScaler().fit_transform(df[["income", "age"]])
```

Categorical features must be converted to numbers for most algorithms:

```
# One-hot encoding: city -> city_mumbai, city_delhi, ...
df = pd.get_dummies(df, columns=["city"], prefix="city")

# Ordinal encoding for ordered categories
from sklearn.preprocessing import OrdinalEncoder
df["education"] = OrdinalEncoder().fit_transform(df[["education"]])
```

Save Clean Data Separately

Keep raw data untouched and write the cleaned version to a new file. This preserves provenance and lets you re-run cleaning when you improve your logic.

```
df.to_parquet("data/clean/customers.parquet")
# or df.to_csv("data/clean/customers.csv", index=False)
```


Practice Task: Take any messy public dataset (a CSV with missing values works well). Apply the full checklist: inspect, handle missing values with a documented strategy, dedupe, fix types and strings, flag outliers (don't delete), and save the cleaned result. Write down every decision you made in a notebook markdown cell.

5. Exploratory Data Analysis

CHAPTER 5

What is EDA?

Exploratory data analysis (EDA) is the disciplined process of getting to know your data before modeling: summarizing it, finding patterns, spotting problems, and generating hypotheses. EDA answers "*what is going on here?*" so that modeling answers "*how do I use it?*".

Summary Statistics

```
import pandas as pd
df = pd.read_parquet("data/clean/customers.parquet")

# Numeric summary
print(df.describe())

# Central tendency
print(df["spend"].mean(), df["spend"].median(), df["spend"].mode())

# Spread
print(df["spend"].std(), df["spend"].var())
print(df["spend"].quantile([0.25, 0.5, 0.75]))

# Counts for categoricals
print(df["city"].value_counts(normalize=True))
```

Mean vs median: the median resists outliers. When income and spend are skewed (a few very large values), the median often describes the "typical" customer better than the mean. Always report both.

Skewness & Distributions

```
skew = df["spend"].skew()
print(skew) # > 1: heavily right-skewed; < -1: left-skewed

# Compare distribution to a normal (histogram + KDE)
import matplotlib.pyplot as plt
import seaborn as sns
sns.histplot(df["spend"], kde=True)
plt.title("Distribution of Spend")
plt.show()
```

Right-skewed data is extremely common in business (spending, wait times, incomes). Common fixes include log transforms (`np.log1p(df["spend"])`) before modeling.

Grouped Analysis (Split-Apply-Combine)

Compare segments — this is where most actionable insight lives.

```
# Average spend by city
print(df.groupby("city")["spend"].agg(["mean", "median", "count"]))

# Spend by city and plan
print(df.groupby(["city", "plan"])["spend"].mean().unstack())

# Cross-tabulation of two categoricals
print(pd.crosstab(df["plan"], df["is_churned"], normalize="index"))
```

Correlations

```
# Pairwise Pearson correlation
corr = df[["spend", "age", "tenure", "support_tickets"]].corr()
print(corr)

# Rank correlation (robust to outliers / monotonic but non-linear)
print(df[["spend", "age"]].corr(method="spearman"))
```

Correlation is not causation. A correlation between two columns may be confounded by a third variable, may be pure chance, or may run in a direction you cannot see from the data alone. EDA finds candidates; experiments (Chapter 8) establish cause.

Comparing Distributions Across Groups

```
# Box plots: distribution by category
sns.boxplot(x="plan", y="spend", data=df)

# Violin plots show the full shape
sns.violinplot(x="plan", y="spend", data=df)
```

Hypothesis Generation in EDA

Every pattern you find should be turned into a testable claim:

- Finding: "Enterprise-plan customers spend 2x more than Basic."
- Question: Is the difference real or due to sample size/outliers?
- Design: A t-test (Chapter 8) between the two groups.

EDA Workflow

1. Profile: shape, types, missingness, duplicates.
2. Univariate: distributions of each column.
3. Bivariate: relationships and segment comparisons.
4. Multivariate: correlations, cross-tabs, interactions.
5. Document: capture findings and questions in your notebook.

Practice Task: On your cleaned dataset, write an EDA notebook that (1) reports key summary statistics, (2) plots the distribution of the target column, (3) compares the target across two or more groups, (4) reports a correlation matrix, and (5) states three concrete hypotheses with the test you would run for each. This notebook is the centerpiece of your portfolio.

6. Data Visualization

CHAPTER 6

Why Visualization is a Core Skill

A good chart compresses thousands of rows into one clear story. Visualization is used for **exploration** (finding patterns yourself) and **communication** (convincing others). The best charts are honest and simple: they reveal the data, not the tooling.

Chart Selection Guide

What you want to show	Best chart
Distribution of one numeric variable	Histogram, box plot, violin plot, KDE
Counts of a category	Bar chart (ordered)
Change over time	Line chart
Relationship between two numerics	Scatter plot (add regression line)
Numeric vs categorical (compare groups)	Box plot, violin plot, bar of means
Part of a whole	Stacked bar (avoid pie > 5 slices)
Two categoricals	Heatmap of counts / stacked bars
Correlation matrix	Correlation heatmap
Geographic data	Choropleth / scatter on map

matplotlib Basics

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_parquet("data/clean/customers.parquet")

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(df["month"], df["revenue"], marker="o", linewidth=2)
ax.set_title("Monthly Revenue")
ax.set_xlabel("Month")
ax.set_ylabel("Revenue (₹)")
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

Style tip: set a consistent style once for the notebook so every chart shares the same fonts and palette:

```
plt.rcParams.update({
    "figure.dpi": 120,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 11
})
```

seaborn for Statistical Charts

seaborn sits on matplotlib and makes statistical charts one line:

```
import seaborn as sns

# Distribution
sns.histplot(df["spend"], kde=True)

# Relationship by group
sns.scatterplot(x="age", y="spend", hue="plan", data=df)

# Compare groups
sns.boxplot(x="plan", y="spend", data=df)

# Correlation heatmap
sns.heatmap(df.select_dtypes("number").corr(), annot=True, cmap="RdBu_r")
```

interactive with plotly

For dashboards and exploration, interactive charts (hover, zoom, filter) add a lot:

```
import plotly.express as px

fig = px.scatter(df, x="age", y="spend", color="plan",
                title="Spend by age and plan")
fig.show()

# Animated line over time
px.line(df, x="month", y="revenue", color="plan")
```

Design Principles

- **Label everything.** Title, axes, units, and legend must be self-explanatory.
- **Start axes at zero** unless there is a reason not to — and if you truncate, say so.
- **Order bars** by value; sort categorical bars so the eye reads the pattern fast.
- **Limit colors.** Use one accent color against neutrals; never rely on color alone (accessibility).
- **Avoid pie charts** for many categories — humans judge areas poorly.
- **Remove clutter:** unnecessary gridlines, borders, and 3D effects hide the data.
- **Honesty:** never cherry-pick time ranges, drop outliers silently, or use a misleading axis.

Common deception: a truncated y-axis can make a 2% change look like a 100% change. In your own charts, let the scale tell the truth; when reading others' charts, check the axis first.

Dashboards & Reports

Beyond notebooks, dashboards share insights with stakeholders:

- **Streamlit** — build data apps in pure Python (recommended starting point).
- **Panel / Dash** — flexible Python dashboards.
- **Metabase / Superset** — SQL-first BI tools.
- **Looker Studio** — free Google BI with BigQuery integration.

Practice Task: Create a small dashboard (Streamlit is ideal) for your cleaned dataset with: one time-series line chart, one grouped comparison, and one correlation heatmap. Then write a two-paragraph narrative explaining the top three insights and add a title and labeled axes to each chart.

7. Statistics & Probability

CHAPTER 7

The Two Branches of Statistics

- **Descriptive statistics** — summarizing what you observed (means, spreads, distributions). You used these in Chapter 5.
- **Inferential statistics** — using a sample to say something about the larger population, with a quantified level of uncertainty. This is the theory under every A/B test and every model evaluation.

Descriptive Statistics in Practice

```
import numpy as np
import pandas as pd
from scipy import stats

spend = np.array([120, 250, 480, 180, 610, 220, 90, 330])

print("mean  :", np.mean(spend))
print("median :", np.median(spend))
print("std   :", np.std(spend, ddof=1)) # sample std
print("IQR   :", np.percentile(spend, 75) - np.percentile(spend, 25))
print("skew  :", stats.skew(spend))
print("kurt  :", stats.kurtosis(spend))
```

Probability Essentials

Concept	Idea	Example
Event	Outcome or set of outcomes	"The ad is clicked"
$P(A)$	Probability of event A	$P(\text{click}) = 0.04$
$P(A B)$	Probability of A given B	$P(\text{click} \mid \text{shown to premium users})$
Independence	Knowing B tells nothing about A	Coin flips
Bayes' rule	Update beliefs with evidence	$P(\text{disease} \mid \text{positive test})$

Bayes' rule is the workhorse of inference:

$$P(A|B) = P(B|A) * P(A) / P(B)$$

Example: a test for a disease is 99% accurate. 1% of people have the disease. If you test positive, what is the chance you have it?


```

P(disease)      = 0.01
P(pos | disease) = 0.99
P(pos | healthy) = 0.01
P(pos) = 0.99*0.01 + 0.01*0.99 = 0.0198
P(disease | pos) = 0.99*0.01 / 0.0198 = 0.5    # only 50%!

```

Lesson: a "99% accurate" test on a rare condition gives a positive result that is still only 50% likely to be real. Base rates matter — this confusion causes real-world errors in medicine, hiring, and fraud detection.

Key Distributions

Distribution	Used for	Shape
Binomial(n, p)	Count of successes in n trials	Discrete, bounded 0..n
Poisson(λ)	Counts of rare events over time/space	Discrete, right-skewed
Normal(μ , σ)	Many natural sums; basis of much inference	Bell curve
Uniform(a, b)	Complete randomness	Flat
Exponential(λ)	Time between events	Decaying

```

# Simulate a binomial: 1000 users, each converts with p=0.05
np.random.seed(42)
conversions = np.random.binomial(n=1000, p=0.05, size=10000)
print(conversions.mean(), conversions.std())
# ~50 and ~6.9 – matches theory: mean=np, std=sqrt(np(1-p))

```

Sampling & the Central Limit Theorem

The **Central Limit Theorem** says: if you take many samples from a population (any distribution, finite variance) and compute their means, the distribution of those sample means approaches a normal distribution as sample size grows. This is why we can compute confidence intervals and p-values with normal-based formulas even when the underlying data is skewed.

```

# Demonstrate the CLT: means of samples from an exponential
sample_means = [np.mean(np.random.exponential(scale=2, size=50))
                 for _ in range(5000)]
print(np.mean(sample_means), np.std(sample_means, ddof=1))
# ~2.0 and ~0.28 = sigma/sqrt(n) = 2/sqrt(50)

```

Standard Error & Confidence Intervals

A confidence interval for a mean is $\text{sample_mean} \pm z * SE$, where $SE = \text{sample_std} / \text{sqrt}(n)$.

```
data = np.random.normal(loc=100, scale=15, size=400)
n = len(data)
se = np.std(data, ddof=1) / np.sqrt(n)
z = stats.norm.ppf(0.975)          # 1.96 for a 95% CI
lo, hi = np.mean(data) - z*se, np.mean(data) + z*se
print(f"95% CI: ({lo:.1f}, {hi:.1f})")
```

Common misinterpretation: a 95% confidence interval is *not* "95% chance the true value is inside." It means: if you repeated the experiment many times, about 95% of the intervals would contain the true value. It is a statement about the procedure, not about any single interval.

Choosing Between Parametric and Non-Parametric

Parametric tests (t-test, ANOVA) assume distributions (usually normality) and use their parameters. Non-parametric tests (Mann-Whitney U, Kruskal-Wallis, bootstrap) make weaker assumptions and are safer for small or skewed samples — at the cost of a little power. With large samples, the difference usually disappears.

Practice Task: Using your dataset, (1) report the distribution shape and skewness of a key column, (2) compute a 95% CI for its mean, (3) demonstrate the CLT on a skewed variable by drawing 5,000 samples and plotting the histogram of sample means, and (4) write a one-line interpretation of the CI in plain business language.

8. Hypothesis Testing & A/B Testing

CHAPTER 8

From Question to Hypothesis

A hypothesis test answers: "*Is the difference I see in my sample real, or could it be chance?*" The machinery:

- **Null hypothesis (H_0):** no effect — e.g., "the new checkout converts at the same rate as the old one."
- **Alternative hypothesis (H_1):** there is an effect — "the new checkout converts at a different rate."
- **Test statistic:** a number computed from the data whose distribution under H_0 is known.
- **p-value:** the probability of seeing a statistic at least as extreme as yours, *if H_0 were true*.

If the p-value is small (below your threshold, usually 0.05), you **reject H_0** — the data are unlikely under "no effect," so you act as if there is an effect.

p-value myths: the p-value is *not* the probability that H_0 is true, and it is *not* the probability that you made a mistake. It is purely conditional on H_0 . "Statistically significant" says nothing about practical importance — a tiny effect can be "significant" with a large sample.

Type I and Type II Errors

	H_0 is true	H_0 is false
You reject H_0	Type I error (false positive)	Correct decision
You fail to reject	Correct decision	Type II error (false negative)

- α = probability of Type I error (significance level, commonly 0.05).
- β = probability of Type II error; **power** = $1 - \beta$.

A t-test in Python

```
from scipy import stats
import numpy as np

# Two independent samples
new = np.random.normal(loc=3.2, scale=1.1, size=200)
old = np.random.normal(loc=3.0, scale=1.1, size=200)

t_stat, p_value = stats.ttest_ind(new, old, equal_var=False)
print(f"t = {t_stat:.3f}, p = {p_value:.4f}")
# p < 0.05 -> reject H0: the means differ
```

Which Test When

Question	Test
Mean of one group vs a value	One-sample t-test
Means of two independent groups	Independent t-test
Paired observations (before/after)	Paired t-test
Two means, non-normal data	Mann-Whitney U
Three+ group means	ANOVA (then post-hoc)
Association of two categoricals	Chi-square test of independence
Correlation significance	Pearson/Spearman + its p-value

A/B Testing in Practice

An A/B test is just a hypothesis test run on a real system: randomly split users into a **control** (old version) and **treatment** (new version), measure a key metric, and compare.

Design checklist:

1. **Choose one primary metric** (e.g., conversion rate). Pre-decide it before the test.
2. **Randomize at the user level** (not the session level — sessions are not independent).
3. **Run a power analysis** to size the experiment before starting.
4. **Run to completion.** Never stop early because results "look done" — early stopping inflates false positives.
5. **Watch for peeking** and repeated testing — every look uses up your error budget.
6. **Check for interference** between groups and novelty effects.

```
# Power analysis to size an A/B test (conversion-rate example)
from statsmodels.stats.power import TTestIndPower

# Baseline 10% -> treatment 12%, alpha 0.05, power 0.8
effect = abs(0.12 - 0.10) / np.sqrt(0.10 * 0.90)
n = TTestIndPower().solve_power(effect_size=effect,
                                alpha=0.05, power=0.8)
print(f"per group ~{int(n)} users")
```

Bootstrap as a robust alternative: when you doubt normality assumptions, you can bootstrap the difference of means and read the confidence interval directly — no formulas required:

```
rng = np.random.default_rng(42)
diffs = []
for _ in range(2000):
    s1 = rng.choice(new, size=len(new), replace=True)
    s2 = rng.choice(old, size=len(old), replace=True)
    diffs.append(s1.mean() - s2.mean())
lo, hi = np.percentile(diffs, [2.5, 97.5])
print(f"95% bootstrap CI of difference: ({lo:.3f}, {hi:.3f})")
```

If the interval excludes zero, the difference is real.

Multiple Comparisons

If you test 20 hypotheses at $\alpha = 0.05$, you expect about 1 false positive by chance. Corrections control this:

- **Bonferroni:** divide α by the number of tests (simple, conservative).
- **Benjamini-Hochberg (FDR):** controls the expected fraction of false discoveries — standard in modern practice.

Practice Task: In your EDA notebook you generated hypotheses. Now run the appropriate test for each: compare two groups with a t-test (or Mann-Whitney), check a categorical association with chi-square, and compute a bootstrap CI for the difference. State for each: the hypothesis, the p-value, whether you reject H_0 , and a plain-language conclusion.

9. SQL for Data Science

CHAPTER 9

Why Data Scientists Need SQL

Most business data lives in relational databases and warehouses. Even with pandas skills, you will reach for SQL because the data is too large to copy, or because it lives behind a warehouse that only answers SQL. SQL is the most transferable data skill — every BI tool, warehouse, and many pipelines speak it.

Core Query Shape

```
SELECT city, COUNT(*) AS customers, AVG(spend) AS avg_spend
FROM customers
WHERE plan = 'enterprise'
GROUP BY city
HAVING AVG(spend) > 500
ORDER BY avg_spend DESC
LIMIT 10;
```

Execution order (conceptual): **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **ORDER BY** → **LIMIT**. This explains why you cannot use an alias defined in **SELECT** inside **WHERE** but can in **ORDER BY**.

Filtering & NULLs

```
SELECT * FROM orders
WHERE created_at >= '2026-01-01'
  AND status IN ('paid', 'shipped')
  AND discount IS NOT NULL;      -- NULLs need IS NULL / IS NOT NULL
```

Joins

Join	Rows returned
INNER JOIN	Only rows that match in both tables
LEFT JOIN	All rows from the left table, matched rows from the right (NULL where missing)
RIGHT JOIN	All rows from the right table (mirror of LEFT)
FULL OUTER JOIN	All rows from both

```
SELECT o.order_number, o.total, c.full_name
FROM orders o
LEFT JOIN customers c ON c.id = o.customer_id
WHERE c.full_name IS NULL;      -- orders with no customer
```

Cardinality check: a JOIN that multiplies rows (one-to-many joined to another one-to-many) inflates aggregates. After joining, verify `COUNT(*)` against expectations before trusting your numbers.

Aggregation

```
SELECT
  date_trunc('month', created_at) AS month,
  COUNT(*)                        AS orders,
  SUM(total)                      AS revenue,
  AVG(total)                     AS avg_order,
  MIN(total)                     AS min_order,
  MAX(total)                     AS max_order
FROM orders
GROUP BY 1
ORDER BY 1;
```

Window Functions

Window functions compute over a set of rows *without collapsing them* — perfect for rankings, running totals, and comparisons.

```
SELECT
  order_number, customer_id, total,
  ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY created_at) AS order_num,
  SUM(total) OVER (PARTITION BY customer_id)                      AS lifetime_value,
  LAG(total) OVER (PARTITION BY customer_id ORDER BY created_at) AS prev_order,
  total - LAG(total) OVER (PARTITION BY customer_id ORDER BY created_at) AS delta,
  RANK() OVER (ORDER BY total DESC)                               AS rank
FROM orders;
```

CTEs (Common Table Expressions)

CTEs make complex queries readable and reusable:

```
WITH cohort AS (
  SELECT customer_id, MIN(created_at)::date AS first_order_date
  FROM orders GROUP BY customer_id
)
SELECT
  date_trunc('month', c.first_order_date) AS cohort_month,
  COUNT(DISTINCT c.customer_id)          AS customers,
  COUNT(DISTINCT o.customer_id)          AS retained
FROM cohort c
LEFT JOIN orders o ON o.customer_id = c.customer_id
  AND date_trunc('month', o.created_at) = date_trunc('month', c.first_order_date) + INTERVAL '1 month'
GROUP BY 1;
```

SQL vs pandas

SQL	pandas
SELECT col	df[["col"]]
WHERE	df[df["col"] > 5]
GROUP BY	df.groupby(...).agg(...)
ORDER BY	df.sort_values(...)
JOIN	df.merge(...)
WINDOW	df.groupby(...).transform / .rank()
DISTINCT	df.drop_duplicates()

Querying with pandas-sql combo

```
import sqlite3, pandas as pd
conn = sqlite3.connect("data/app.db")

# pull a subset to pandas, then analyze
df = pd.read_sql_query("""
    SELECT c.city, o.total, o.created_at
    FROM orders o JOIN customers c ON c.id = o.customer_id
    WHERE o.created_at >= '2025-01-01'
""", conn)
conn.close()
```

BigQuery in the browser: for large public data, Google BigQuery's free tier lets you practice SQL on real datasets (e.g., GA4 sample data, GitHub commits) with no setup — an excellent playground to master warehouse SQL and window functions.

Query Performance Habits

- Filter (`WHERE`) before joining and aggregating where possible.
- Only `SELECT` the columns you need.
- Use `EXPLAIN` on warehouses to see the plan.
- In BigQuery, partition and cluster tables by the columns you filter most.
- Prefer `COUNT(DISTINCT ...)` over joining to dedupe.

Practice Task: Load any public dataset into SQLite, then write and run: (1) a query with a JOIN, (2) a monthly aggregate with window functions for running totals, and (3) a cohort retention CTE. Compare the result of query (1) with the pandas equivalent using `merge`.